

Методичка 3. Цикл while

1. Зачем нужна эта тема

Цикл нужен, когда одно и то же действие нужно выполнить несколько раз.

Без циклов пришлось бы писать одинаковые команды вручную много раз.

Например, без цикла:

```
print("Привет")
print("Привет")
print("Привет")
print("Привет")
print("Привет")
```

С циклом это можно сделать короче и удобнее.

Цикл `while` особенно полезен, когда заранее неизвестно, сколько раз нужно повторять действие.

Например:

- пользователь вводит числа, пока не введёт `0` ;
- игрок угадывает число, пока не даст правильный ответ;
- программа спрашивает пароль, пока он не станет верным;
- меню работает, пока пользователь не выберет выход;
- программа повторяет ввод, пока данные не станут корректными.

Эта тема связана с условиями. Цикл `while` тоже использует условие, но не просто проверяет его один раз, а повторяет блок кода, пока условие истинно.

2. Главная идея темы

Цикл `while` означает “пока”.

Программа выполняет блок кода снова и снова, пока условие остаётся истинным.

Цикл — повторение действий.

Итерация — один проход цикла.

Условие цикла — проверка, от которой зависит, продолжится цикл или остановится.

Счётчик — переменная, которая считает количество повторений.

Накопитель — переменная, которая постепенно собирает результат, например сумму.

3. Базовый синтаксис

Простой цикл `while`

```
count = 1

while count <= 5:
    print("Привет")
    count = count + 1
```

Что происходит:

- `count` начинается с 1;
- программа проверяет `count <= 5`;
- если условие истинно, выводит `Привет`;
- увеличивает `count`;
- снова проверяет условие;
- когда `count` станет 6, цикл остановится.

На что обратить внимание:

- условие пишется после `while`;
- после условия ставится двоеточие;
- тело цикла пишется с отступом;
- внутри цикла должно происходить изменение, которое когда-нибудь остановит цикл.

Цикл с вводом

```
password = ""

while password != "1234":
    password = input("Введите пароль: ")

print("Вход выполнен")
```

Программа будет спрашивать пароль, пока пользователь не введёт `1234` .

Цикл с суммой

```
number = 1
total = 0

while number != 0:
    number = int(input("Введите число: "))
    total = total + number

print("Сумма:", total)
```

Программа вводит числа и складывает их, пока пользователь не введёт 0.

4. Разбор темы от простого к сложному

Шаг 1. Повторить действие несколько раз

```
i = 1

while i <= 3:
    print("Python")
    i = i + 1
```

Программа выводит слово `Python` три раза.

Здесь `i` — счётчик.

Шаг 2. Вывести числа от 1 до 5

```
i = 1

while i <= 5:
    print(i)
    i = i + 1
```

Программа выводит:

```
1
2
3
4
5
```

Сначала печатается текущее значение `i`, потом `i` увеличивается.

Шаг 3. Посчитать сумму чисел от 1 до 5

```
i = 1
total = 0

while i <= 5:
    total = total + i
    i = i + 1

print("Сумма:", total)
```

Что происходит:

- `total` начинается с 0;
- на каждом шаге к нему прибавляется `i`;
- после цикла выводится сумма.

Шаг 4. Вводить данные до специального значения

```
number = int(input("Введите число: "))

while number != 0:
    print("Вы ввели:", number)
    number = int(input("Введите число: "))

print("Конец программы")
```

Цикл работает, пока пользователь не введёт 0.

Шаг 5. Проверка пароля

```
password = input("Введите пароль: ")

while password != "python":
    print("Неверный пароль")
    password = input("Введите пароль: ")

print("Доступ разрешён")
```

Здесь заранее неизвестно, сколько раз пользователь ошибётся. Поэтому подходит `while`.

Шаг 6. Игровой цикл

```
secret = 7
answer = int(input("Угадайте число: "))

while answer != secret:
    print("Не угадали")
    answer = int(input("Попробуйте ещё раз: "))

print("Победа!")
```

Цикл продолжается, пока игрок не угадает число.

Шаг 7. Меню программы

```
command = ""

while command != "0":
    print("1 — Поздороваться")
    print("2 — Прощаться")
    print("0 — Выход")

    command = input("Выберите команду: ")

    if command == "1":
        print("Привет!")
    elif command == "2":
        print("Пока!")

print("Программа завершена")
```

Так строятся простые консольные меню.

5. Частые ошибки учеников

Ошибка 1. Бесконечный цикл

Неправильно:

```
i = 1

while i <= 5:
    print(i)
```

Почему это ошибка:

`i` никогда не меняется. Условие всегда истинно, поэтому цикл не остановится.

Правильно:

```
i = 1

while i <= 5:
    print(i)
    i = i + 1
```

Ошибка 2. Неправильное начальное значение

Неправильно:

```
i = 10

while i <= 5:
    print(i)
    i = i + 1
```

Почему это ошибка:

Условие `i <= 5` сразу ложно. Цикл не выполнится ни разу.

Правильно:

```
i = 1

while i <= 5:
    print(i)
    i = i + 1
```

Ошибка 3. Забыли отступ

Неправильно:

```
i = 1

while i <= 5:
print(i)
i = i + 1
```

Почему это ошибка:

Python не понимает, какие строки относятся к циклу.

Правильно:

```
i = 1

while i <= 5:
    print(i)
    i = i + 1
```

Ошибка 4. Обновляют счётчик не в том месте

Неправильно:

```
i = 1

while i <= 5:
    i = i + 1
    print(i)
```

Почему это ошибка:

Программа сначала увеличивает `i`, а потом печатает. Поэтому вывод начнётся с 2.

Правильно:

```
i = 1

while i <= 5:
    print(i)
    i = i + 1
```

Ошибка 5. Неправильно используют накопитель

Неправильно:

```
i = 1

while i <= 5:
    total = 0
    total = total + i
    i = i + 1

print(total)
```

Почему это ошибка:

`total` каждый раз снова становится 0. Сумма не накапливается.

Правильно:

```
i = 1
total = 0

while i <= 5:
    total = total + i
    i = i + 1

print(total)
```

Ошибка 6. Путают условие продолжения и условие остановки

Неправильно:

```
password = ""

while password == "1234":
    password = input("Пароль: ")

print("Вход")
```

Почему это ошибка:

Цикл должен работать, пока пароль неправильный. А здесь он работает только если пароль уже правильный.

Правильно:


```
password = ""

while password != "1234":
    password = input("Пароль: ")

print("Вход")
```

Ошибка 7. Забывают получить новое значение внутри цикла

Неправильно:

```
number = int(input("Введите число: "))

while number != 0:
    print("Вы ввели:", number)
```

Почему это ошибка:

Пользователь вводит число только один раз. Если число не 0, цикл станет бесконечным.

Правильно:

```
number = int(input("Введите число: "))

while number != 0:
    print("Вы ввели:", number)
    number = int(input("Введите число: "))
```

6. Мини-примеры для закрепления

Пример 1. Вывод чисел

```
i = 1

while i <= 5:
    print(i)
    i = i + 1
```

Программа выводит числа от 1 до 5. Закрепляется счётчик.

Пример 2. Обратный отсчёт

```
i = 5

while i >= 1:
    print(i)
    i = i - 1

print("Старт!")
```

Программа считает от 5 до 1. Закрепляется уменьшение счётчика.

Пример 3. Сумма чисел

```
i = 1
total = 0

while i <= 10:
    total = total + i
    i = i + 1

print(total)
```

Программа считает сумму чисел от 1 до 10. Закрепляется накопитель.

Пример 4. Ввод до нуля

```
number = int(input("Введите число: "))

while number != 0:
    print("Число:", number)
    number = int(input("Введите число: "))

print("Готово")
```

Программа работает, пока пользователь не введёт 0. Закрепляется цикл с условием остановки.

Пример 5. Количество попыток

```
password = ""
attempts = 0

while password != "admin":
    password = input("Пароль: ")
    attempts = attempts + 1

print("Попыток:", attempts)
```

Программа считает, сколько раз пользователь вводил пароль. Закрепляется счётчик внутри цикла.

Пример 6. Сумма введенных чисел

```
number = int(input("Введите число: "))
total = 0

while number != 0:
    total = total + number
    number = int(input("Введите число: "))

print("Сумма:", total)
```

Программа складывает все числа до ввода 0. Закрепляется накопитель и ввод в цикле.

Пример 7. Простое меню

```
command = ""

while command != "0":
    command = input("Введите команду 1 или 0: ")

    if command == "1":
        print("Команда выполнена")

print("Выход")
```

Программа работает, пока пользователь не выберет выход. Закрепляется меню на `while`.

7. Практические задания

Лёгкий уровень

1. Вывести числа от 1 до 10 с помощью `while` .
2. Вывести числа от 10 до 1 с помощью `while` .
3. Вывести слово `Python` 5 раз.
4. Посчитать сумму чисел от 1 до 100.
5. Вывести все чётные числа от 2 до 20.

Средний уровень

1. Пользователь вводит числа, пока не введёт 0. Вывести количество введенных чисел.
2. Пользователь вводит числа, пока не введёт 0. Вывести сумму всех чисел.
3. Пользователь вводит пароль, пока не введёт правильный.
4. Сделать программу, которая просит ввести положительное число. Если введено отрицательное, просит снова.
5. Сделать простое меню: `1 — привет` , `2 — помощь` , `0 — выход` .

Повышенный уровень

1. Пользователь загадывает число в голове, а программа просит вводить команды `больше` , `меньше` , `угадал` . Реализовать цикл до команды `угадал` .
2. Написать программу, которая считает среднее арифметическое введенных чисел до 0.
3. Сделать ограничение на 3 попытки ввода пароля.

8. Контрольные вопросы

1. Что означает слово `while` ?
2. Чем цикл отличается от условия?
3. Что такое одна итерация цикла?
4. Зачем нужен счётчик?
5. Зачем нужен накопитель?
6. Почему может возникнуть бесконечный цикл?
7. Что должно изменяться внутри цикла?
8. Когда цикл `while` удобнее, чем просто повторение кода?
9. Почему важно правильно выбрать начальное значение счётчика?
10. Что произойдёт, если условие цикла сразу ложно?
11. Как сделать ввод чисел до 0?
12. Как построить простое меню через `while` ?

9. Краткий конспект в конце

Главная идея темы: цикл `while` повторяет блок кода, пока условие истинно.

Основная конструкция:

```
while условие:  
    команды
```

Пример со счётчиком:

```
i = 1  
  
while i <= 5:  
    print(i)  
    i = i + 1
```

Важно запомнить:

- после `while` ставится условие;
- после условия ставится двоеточие;
- тело цикла пишется с отступом;
- внутри цикла должно быть изменение, которое поможет циклу остановиться;
- счётчик считает повторы;
- накопитель собирает результат.

Частые ошибки:

- бесконечный цикл;
- забыть изменить счётчик;
- неправильно поставить начальное значение;
- обновлять переменную не в том месте;
- каждый раз обнулять накопитель внутри цикла.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- писать цикл `while`;
- использовать счётчик;
- использовать накопитель;
- выводить последовательность чисел;
- повторять ввод до нужного значения;

- делать простое меню;
- находить бесконечные циклы;
- исправлять ошибки в условиях цикла;
- решать задачи, где количество повторений заранее неизвестно.